The Master Theorem

Outline

Recurrences in standard form

The Master Theorem

Applying the Master Theorem

When the Master Theorem does not apply

Proof of the Master Theorem

Introduction (1/2)

- In the chapter *Recurrences* we solved recurrences by expansion, by substitution and with recursion trees
- Each method requires some computation, and for divide-and-conquer recurrences the computation is always the same:
  - draw the tree
  - sum the level costs
  - compare the root with the leaves
- At the end of that chapter we observed three possible behaviours: the **leaves** dominate, all the **levels** cost the same, or the **root** dominates

Introduction (2/2)

The **Master Theorem** turns these three behaviours into a ready-to-use result. In this chapter we

- recall the recurrences in standard form and the algorithms that produce them
- state the theorem and explain its three cases
- apply it with a fixed five-step scheme to many examples, and show the situations in which it cannot be applied
- prove it, first for exact powers of b and then for every natural number

Recurrences in standard form

Introduction: Recurrences in standard form

- In this section we recall the shape of the recurrences produced by divide-and-conquer algorithms: the **standard form**
- We name its three ingredients: the number of subproblems a, the factor b that reduces the size, and the cost $f(n)$ of dividing and combining
- These three ingredients are all the Master Theorem needs to give the order of growth of the solution

Recurrences

- In the chapter *Recurrences* we have seen that the number of steps executed by a recursive algorithm is naturally described by a **recurrence**
- A recurrence is an equation that expresses the value of a function on an input of size n in terms of its values on smaller inputs
- A large family of *divide-and-conquer* algorithms produces recurrences of a very specific shape, called the **standard form**:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

The standard form

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n)$$

where:

- n is the size of the input;
- a is the **number of subproblems** generated by each recursive call ($a \geq 1$, constant);
- $\frac{n}{b}$ is the **size of each subproblem** ($b > 1$, constant); when n/b is not a natural number it is understood as $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$;
- $f(n)$ is the **cost of dividing** the problem into the a subproblems **and of combining** their solutions (everything that is done outside the recursive calls)

Positivity of $f(n)$ and base case

- We always assume that $f(n)$ is **asymptotically positive**, i.e.,

$$f(n) > 0 \quad \text{for all sufficiently large } n$$

- For sufficiently small n (for example $n = 1$) the recursion stops and

$$T(n) = \Theta(1) :$$

  solving a problem of constant size requires a constant number of steps

- For this reason the base case is usually not even written

A recurrence as a mathematical object

- In this chapter a recurrence is a **mathematical object**: we use a generic function $T(n)$
- When we analyse a recursive algorithm, the function described by the recurrence is the number of steps $S(n)$ executed in the worst case on an input of size parameter n
- The worst-case running time $T(|I|)$ is then obtained from $S(n)$ exactly as in the chapter *Running time*

Examples of recurrences in standard form

They will all be solved later in this chapter:

Algorithm	Recurrence	a	b	$f(n)$
Binary search (recursive)	$T(n) = T(n/2) + 1$	1	2	1
Array sum (recursive)	$T(n) = 2T(n/2) + 1$	2	2	1
Merge sort	$T(n) = 2T(n/2) + n$	2	2	n
Karatsuba multiplication	$T(n) = 3T(n/2) + n$	3	2	n
Strassen matrix mult.	$T(n) = 7T(n/2) + n^2$	7	2	n^2

- The **Master Theorem** gives the order of growth of $T(n)$ for most recurrences in standard form, directly from a, b and $f(n)$, without any computation of the recursion

Summary

Standard form:

- $T(n) = a\,T(n/b) + f(n)$, with constants $a \geq 1$ and $b > 1$
- n/b means $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$ when it is not a natural number
- $f(n)$ is the cost outside the recursive calls, asymptotically positive

Base case:

- $T(n) = \Theta(1)$ for sufficiently small n, usually not written

Examples:

- binary search, array sum, merge sort, Karatsuba, Strassen

The Master Theorem

Introduction: The Master Theorem

- In this section we state the theorem and explain its **three cases**
- We give a rule of thumb to guess the case, and the intuition behind it: the costs of the levels of the recursion tree
- We also see that the three cases leave **gaps**, in which the theorem gives no answer, and why the base of the logarithms matters only in the exponent

The Master Theorem (1/2)

Theorem 1: Master Theorem

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be an asymptotically positive function, and let $T(n)$ be defined on the natural numbers by the recurrence

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n),$$

where n/b is interpreted as either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$, and $T(n) = \Theta(1)$ for sufficiently small n. Then $T(n)$ has the following asymptotic bounds (next slide).

The Master Theorem (2/2)

Theorem: Master Theorem (continued)

Case 1: *if* $f(n) = O\left(n^{\log_b a - \epsilon}\right)$ *for some constant* $\epsilon > 0$, *then* $T(n) = \Theta\left(n^{\log_b a}\right)$;

Case 2: *if* $f(n) = \Theta\left(n^{\log_b a} \log^k n\right)$ *for some constant* $k \geq 0$, *then* $T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right)$;

Case 3: *if* $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ *for some constant* $\epsilon > 0$, *and if*

$$a f\left(\frac{n}{b}\right) \leq c f(n)$$

for some constant $c < 1$ *and all sufficiently large* n (***regularity condition***), *then* $T(n) = \Theta\left(f(n)\right)$.

About the statement

- This is the classical statement of the theorem (as in the textbook of Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*), with the **extended case 2**
- For $k = 0$ we obtain the basic statement:

$$\text{if } f(n) = \Theta(n^{\log_b a}) \text{ then } T(n) = \Theta(n^{\log_b a} \log n)$$

- The notations O, Ω and Θ in the statement are those defined in the chapter «Asymptotic notation»
- The proof is given in the last section of this chapter

Before applying the theorem, we learn how to guess quickly which of the three cases we are in.

Rule of thumb (1/2)

- In each of the three cases we compare the function $f(n)$ with the function

$$n^{\log_b a},$$

  which (as we will see in the proof) is the **number of leaves** of the recursion tree

- **Intuitively, the larger of the two functions determines the solution of the recurrence**

- As a rule of thumb, check the relation between $n^{\log_b a}$ and $f(n)$ (next slide) and determine the order of growth of $T(n)$ accordingly (if possible)

Rule of thumb (2/2)

$$
n^{\log_b a} \begin{cases} \text{polynomially larger than } f(n) \\ \qquad \longrightarrow \quad \text{case 1:} \quad T(n) = \Theta\left(n^{\log_b a}\right) \\ \\ \text{of the same order as } f(n) \text{ (up to } \log^k n) \\ \qquad \longrightarrow \quad \text{case 2:} \quad T(n) = \Theta\left(n^{\log_b a} \log^{k+1} n\right) \\ \\ \text{polynomially smaller than } f(n) \\ \qquad \longrightarrow \quad \text{case 3:} \quad T(n) = \Theta\left(f(n)\right) \end{cases}
$$

What "polynomially" means

- **Polynomially** smaller (larger) means smaller (larger) **by a factor** n^ϵ for some constant $\epsilon > 0$
- For example:
  - n is polynomially smaller than n^2 (factor n^1)
  - n^2 is polynomially smaller than $n^{2.1}$ (factor $n^{0.1}$)
  - n is smaller than $n \log n$, but **not** polynomially smaller: for every $\epsilon > 0$

$$\lim_{n\to\infty} \frac{n \log n}{n^{1+\epsilon}} = \lim_{n\to\infty} \frac{\log n}{n^\epsilon} = 0^+,$$

hence $n \log n$ grows more slowly than $n \cdot n^\epsilon$, whatever $\epsilon > 0$ we choose

Intuition: the costs of the levels (1/2)

- Unfolding the recurrence (with the recursion tree that we will draw in the proof) the total cost is

$$T(n) = \underbrace{\Theta\left(n^{\log_b a}\right)}_{\text{cost of the leaves}} + \underbrace{\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)}_{\text{cost of the internal levels}}$$

  where level j of the tree contains a^j subproblems of size n/b^j

- Consider the simple case $f(n) = n^d$. The cost of level j is

$$a^j \left(\frac{n}{b^j}\right)^d = n^d \left(\frac{a}{b^d}\right)^j,$$

  so the costs of the levels form a **geometric progression with ratio** a/b^d

Intuition: the costs of the levels (2/2)

Since $a/b^d > 1 \iff d < \log_b a$:

- case 1 ($d < \log_b a$): the level costs **increase** geometrically from the root to the leaves; the total is dominated by the leaves, i.e., by $n^{\log_b a}$
- case 2 ($d = \log_b a$): all the $\log_b n$ levels have **the same cost** $n^d = n^{\log_b a}$; the total is $n^{\log_b a}$ times the number of levels, i.e., $n^{\log_b a} \log n$
- case 3 ($d > \log_b a$): the level costs **decrease** geometrically; the total is dominated by the root, i.e., by $f(n)$

The gaps between the cases

- The three cases **do not cover all the possible functions** $f(n)$:
  - a gap between case 1 and case 2 when $f(n)$ is smaller than $n^{\log_b a}$ but **not polynomially smaller** (and not of the form $n^{\log_b a} \log^k n$ with $k \geq 0$), e.g., $f(n) = n^{\log_b a} / \log n$
  - a gap between case 2 and case 3 when $f(n)$ is larger than $n^{\log_b a}$ but **not polynomially larger** (and not of the form $n^{\log_b a} \log^k n$), e.g., $f(n) = n^{\log_b a} 2^{\sqrt{\log_2 n}}$
  - case 3 does not apply also when $f(n)$ is polynomially larger than $n^{\log_b a}$ but the **regularity condition** fails
- In these situations the Master Theorem **cannot be used**, and the recurrence must be solved with other techniques (recursion tree, iteration, substitution method), as we will see later in this chapter

A remark on the base of the logarithms

- Since $\log_c n = \dfrac{\log_2 n}{\log_2 c}$ for every base $c > 1$, we have

$$\log_c^k n = \frac{1}{\log_2^k c} \log_2^k n = \Theta\big(\log_2^k n\big)$$

- Thus, **inside Θ, O and Ω the base of the logarithm is irrelevant**: in case 2 we simply write $\log^k n$

- On the contrary, the base b in the **exponent** $n^{\log_b a}$ is essential: for instance

$$n^{\log_2 4} = n^2 \qquad \text{whereas} \qquad n^{\log_4 4} = n,$$

and $n^2 \neq \Theta(n)$

The Master Theorem in a nutshell

- The Master Theorem compares the cost of the root, $f(n)$, with the number of leaves of the recursion tree, $n^{\log_b a}$: the larger of the two determines $T(n)$
- When they are of the same order (up to $\log^k n$) one more factor $\log n$ appears
- "Larger" and "smaller" must be **polynomial** (by a factor n^ϵ); in case 3 the **regularity condition** must also hold
- When $f(n)$ falls in a gap between the cases, the theorem gives no answer

Summary

The three cases, comparing $f(n)$ with $n^{\log_b a}$:

- case 1, f polynomially smaller: $T(n) = \Theta(n^{\log_b a})$
- case 2, $f = \Theta(n^{\log_b a} \log^k n)$: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- case 3, f polynomially larger and regular: $T(n) = \Theta(f(n))$

Intuition:

- the level costs increase, stay equal, or decrease from the root to the leaves

Limits:

- gaps between the cases; base of the logarithms essential only in $n^{\log_b a}$

Applying the Master Theorem

Introduction: Applying the Master Theorem

- In this section we apply the theorem with a **fixed five-step scheme**
- We start with three recurrences, one for each case, and then solve the recurrences of the algorithms listed in the previous section
- A numerical check confirms that the theorem gives the **order of growth**, not the constant

The five-step scheme

To solve a recurrence with the Master Theorem we always follow this scheme:

1. **identify** a, b and $f(n)$ (and check that $a \geq 1$, $b > 1$ are constants and $f(n)$ is asymptotically positive);
2. **compute** $n^{\log_b a}$;
3. **compare** $n^{\log_b a}$ with $f(n)$ and guess the case;
4. **verify** the hypothesis of the case: find $\epsilon > 0$ (case 1 or 3), find $k \geq 0$ (case 2), and check the regularity condition (case 3);
5. **conclude**.

Example: $T(n) = 16\,T(n/4) + n$

1. $a = 16,\ b = 4,\ f(n) = n$
2. $n^{\log_b a} = n^{\log_4 16} = n^2$
3. n^2 is polynomially larger than $f(n) = n \longrightarrow$ case 1
4. $f(n) = n = O\left(n^{2-\epsilon}\right)$ with $\epsilon = 1$
5. $T(n) = \Theta\left(n^{\log_4 16}\right) = \Theta(n^2)$

Example: $T(n) = 8\,T(n/2) + n^3$

1. $a = 8,\ b = 2,\ f(n) = n^3$
2. $n^{\log_b a} = n^{\log_2 8} = n^3$
3. n^3 and $f(n) = n^3$ have the same order $\longrightarrow$ case 2
4. $f(n) = n^3 = \Theta\left(n^3 \log^k n\right)$ with $k = 0$
5. $T(n) = \Theta\left(n^3 \log^{k+1} n\right) = \Theta(n^3 \log n)$

Example: $T(n) = 8\,T(n/3) + n^2\ (1/2)$

1. $a = 8,\ b = 3,\ f(n) = n^2$
2. $n^{\log_b a} = n^{\log_3 8} \approx n^{1.893}$
3. $n^{1.893}$ is polynomially smaller than $f(n) = n^2 \longrightarrow$ case 3
4. $f(n) = n^2 = \Omega\left(n^{\log_3 8 + \epsilon}\right)$ for every $0 < \epsilon \leq 2 - \log_3 8 \approx 0.107$
   Regularity condition: see the next slide
5. $T(n) = \Theta(n^2)$

Example: $T(n) = 8\,T(n/3) + n^2$ (2/2)

Regularity condition (step 4): we look for a constant $c < 1$ such that, for all large n,

$$8\,f\left(\frac{n}{3}\right) \leq c\,f(n)$$

$$\iff \quad 8\left(\frac{n}{3}\right)^2 \leq c\,n^2$$

$$\iff \quad \frac{8}{9}\,n^2 \leq c\,n^2,$$

which holds for every c with $\frac{8}{9} \leq c < 1$, e.g., $c = \frac{8}{9}$

Hence case 3 applies and $T(n) = \Theta(n^2)$

Now we solve the recurrences of the algorithms listed at the beginning of the chapter.

Example: binary search, $T(n) = T(n/2) + 1$

1. $a = 1,\ b = 2,\ f(n) = 1$
2. $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$
3. 1 and $f(n) = 1$ have the same order $\longrightarrow$ case 2
4. $f(n) = 1 = \Theta\left(n^0 \log^k n\right)$ with $k = 0$
5. $T(n) = \Theta\left(n^0 \log n\right) = \Theta(\log n)$

Example: merge sort, $T(n) = 2\,T(n/2) + n$

1. $a = 2,\ b = 2,\ f(n) = n$
2. $n^{\log_b a} = n^{\log_2 2} = n$
3. n and $f(n) = n$ have the same order $\longrightarrow$ case 2
4. $f(n) = n = \Theta\left(n \log^k n\right)$ with $k = 0$
5. $T(n) = \Theta(n \log n)$

Example: array sum, $T(n) = 2\,T(n/2) + 1$ (1/2)

1. $a = 2,\ b = 2,\ f(n) = 1$
2. $n^{\log_b a} = n^{\log_2 2} = n$
3. n is polynomially larger than $f(n) = 1 \longrightarrow$ case 1
4. $f(n) = 1 = O\left(n^{1-\epsilon}\right)$ with $\epsilon = 1$ (indeed $n^{1-1} = n^0 = 1$)
5. $T(n) = \Theta(n)$

Example: array sum, $T(n) = 2\,T(n/2) + 1$ (2/2)

- The recursive sum of an array that splits the array into two halves (the algorithm `RecursiveSum` of the chapter «Recursive algorithms») is an algorithm of this kind
- Each call does a constant amount of work outside the recursive calls, and the total number of calls is $\Theta(n)$
- Counting them (in the Colab notebooks of the chapter), on $n = 8$, 1000 and $2^{20} = 1\,048\,576$ entries we obtain 15, 1 999 and 2 097 151 calls, that is, exactly $2n - 1$

Run in Colab: Python · C++ · Java

Example: Karatsuba, $T(n) = 3\,T(n/2) + n$

1. $a = 3,\ b = 2,\ f(n) = n$
2. $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$
3. $n^{1.585}$ is polynomially larger than $f(n) = n \longrightarrow$ case 1
4. $f(n) = n = O\left(n^{\log_2 3 - \epsilon}\right)$ with $\epsilon = \log_2 3 - 1 \approx 0.585 > 0$
5. $T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right)$: Karatsuba's algorithm is asymptotically faster than the schoolbook $\Theta(n^2)$ multiplication

Example: Strassen, $T(n) = 7\,T(n/2) + n^2$

1. $a = 7,\ b = 2,\ f(n) = n^2$
2. $n^{\log_b a} = n^{\log_2 7} \approx n^{2.807}$
3. $n^{2.807}$ is polynomially larger than $f(n) = n^2 \longrightarrow$ case 1
4. $f(n) = n^2 = O\left(n^{\log_2 7 - \epsilon}\right)$ with $\epsilon = \log_2 7 - 2 \approx 0.807 > 0$
5. $T(n) = \Theta\left(n^{\log_2 7}\right) \approx \Theta\left(n^{2.807}\right)$, better than the $\Theta(n^3)$ of the standard matrix multiplication (whose divide-and-conquer recurrence is $T(n) = 8\,T(n/2) + n^2$)

Two more recurrences with a logarithm in $f(n)$,
and one with a non-integer b.

Example: $T(n) = 2\,T(n/2) + n \log n$

1. $a = 2,\ b = 2,\ f(n) = n \log n$
2. $n^{\log_b a} = n^{\log_2 2} = n$
3. $f(n) = n \log n$ is larger than n, but only by a factor $\log n$, not polynomially $\longrightarrow$ not case 3; it has the form $n^{\log_b a} \log^k n \longrightarrow$ case 2
4. $f(n) = n \log n = \Theta\left(n \log^k n\right)$ with $k = 1$
5. $T(n) = \Theta\left(n \log^2 n\right)$

Example: $T(n) = 3\,T(n/4) + n \log n$ (1/2)

1. $a = 3$, $b = 4$, $f(n) = n \log n$ (logarithm in base 2)
2. $n^{\log_b a} = n^{\log_4 3} \approx n^{0.792}$
3. $n^{0.792}$ is polynomially smaller than $f(n) = n \log n \longrightarrow$ case 3
4. Since $n \log n \geq n$ for $n \geq 2$, we have $f(n) = \Omega\left(n^{\log_4 3 + \epsilon}\right)$ with $\epsilon = 1 - \log_4 3 \approx 0.208$

   Regularity condition: see the next slide
5. $T(n) = \Theta(n \log n)$

Example: $T(n) = 3\,T(n/4) + n \log n$ (2/2)

Regularity condition (step 4): for every $n \geq 4$,

$$3\,f\left(\frac{n}{4}\right) = 3\,\frac{n}{4}\,\log\frac{n}{4} \leq \frac{3}{4}\,n \log n = c\,f(n) \qquad \text{with } c = \frac{3}{4} < 1,$$

because $\log(n/4) \leq \log n$

Hence case 3 applies and $T(n) = \Theta(n \log n)$

Example: $T(n) = T(2n/3) + 1$

1. $a = 1$, $b = \frac{3}{2}$ (indeed $\frac{2n}{3} = \frac{n}{3/2}$), $f(n) = 1$. Note that b does not need to be an integer
2. $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1$
3. 1 and $f(n) = 1$ have the same order $\longrightarrow$ case 2
4. $f(n) = 1 = \Theta\left(n^0 \log^k n\right)$ with $k = 0$
5. $T(n) = \Theta(\log n)$

A numerical check (1/2)

- For each of the recurrences above with b integer we computed $T(n)$ exactly from the recurrence, with $T(1) = 1$ and $n = b^L$
- We divided it by the order of growth $g(n)$ given by the theorem (logarithms in base 2)
- The ratio stays bounded between two positive constants, as the theorem guarantees, and approaches the limit in the last column of the table (next slide)

A numerical check (2/2)

Recurrence	$g(n)$	$L = 5$	$L = 10$	$L = 20$	limit
$16\,T(n/4) + n$	n^2	1.3330	1.3333	1.3333	$4/3$
$8\,T(n/2) + n^3$	$n^3 \log n$	1.2000	1.1000	1.0500	1
$8\,T(n/3) + n^2$	n^2	4.5606	6.5364	8.2414	9
$T(n/2) + 1$	$\log n$	1.2000	1.1000	1.0500	1
$2\,T(n/2) + n$	$n \log n$	1.2000	1.1000	1.0500	1
$2\,T(n/2) + 1$	n	1.9688	1.9990	2.0000	2
$3\,T(n/2) + n$	$n^{\log_2 3}$	2.7366	2.9653	2.9994	3
$7\,T(n/2) + n^2$	$n^{\log_2 7}$	2.2521	2.3284	2.3333	$7/3$
$2\,T(n/2) + n \log n$	$n \log^2 n$	0.6400	0.5600	0.5275	$1/2$
$3\,T(n/4) + n \log n$	$n \log n$	2.1933	2.8704	3.4020	4

Reading the table

- For merge sort, for instance, $T(2^L) = 2^L(L+1)$, so

$$T(n) = n \log_2 n + n \qquad \text{and} \qquad \frac{T(n)}{n \log_2 n} = 1 + \frac{1}{L}$$

- In case 3 the convergence is slow: the ratio tends to $\frac{1}{1-c}$ (9 for $c = \frac{8}{9}$, 4 for $c = \frac{3}{4}$), the bound that appears in the proof of the theorem

- The theorem gives the **order of growth**, not the constant: in the table the ratios tend to different constants, but they never grow or vanish

Summary

Five steps:

- identify $a, b, f(n)$; compute $n^{\log_b a}$; compare and guess the case; verify (ϵ, k, regularity); conclude

Results:

- case 1: $16T(n/4) + n$, array sum, Karatsuba, Strassen
- case 2: $8T(n/2) + n^3$, binary search, merge sort, $2T(n/2) + n \log n$, $T(2n/3) + 1$
- case 3: $8T(n/3) + n^2$, $3T(n/4) + n \log n$

Numerical check:

- $T(n)/g(n)$ stays between two positive constants

When the Master Theorem does not apply

Introduction: When the Master Theorem does not apply

- Before applying the theorem, always check that **all** its hypotheses hold
- In this section we see typical situations in which the theorem **cannot** be applied: a gap between the cases, recurrences not in standard form, a not constant or smaller than 1, a regularity condition that fails
- When possible, we solve the recurrence with another technique

Example: a gap, $T(n) = 2\,T(n/2) + n/\log_2 n$ (1/2)

- $a = 2$, $b = 2$, $f(n) = n/\log_2 n$, and $n^{\log_b a} = n$
- **Not case 1**: $f(n)$ is smaller than n, but not polynomially smaller. For every $\epsilon > 0$

$$\lim_{n\to\infty} \frac{n/\log_2 n}{n^{1-\epsilon}} = \lim_{n\to\infty} \frac{n^{\epsilon}}{\log_2 n} = +\infty,$$

  hence $f(n) \neq O(n^{1-\epsilon})$
- **Not case 2**: $f(n) = n\log^{-1} n$ would require $k = -1 < 0$
- **Not case 3**: $f(n)$ is smaller than n

Example: a gap, $T(n) = 2\,T(n/2) + n/\log_2 n$ (2/2)

Solution with the recursion tree (as in the proof of the theorem), for $n = 2^L$ and $T(1) = 1$: level j costs

$$2^j \frac{n/2^j}{\log_2(n/2^j)} = \frac{n}{L-j},$$

so

$$T(n) = n + \sum_{j=0}^{L-1} \frac{n}{L-j} = n + n\sum_{i=1}^{L} \frac{1}{i} = n\,(1 + H_L),$$

where $H_L = \sum_{i=1}^{L} \frac{1}{i}$ is the L-th harmonic number. Since $\ln(L+1) \le H_L \le 1 + \ln L$ (comparison with $\int \frac{dx}{x}$), $H_L = \Theta(\log L)$ and

$$T(n) = \Theta(n \log\log n)$$

Example: not in standard form, $T(n) = T(n-1) + n$

- The subproblem has size $n-1$, not n/b with a constant $b > 1$: the recurrence is **not in standard form**
- Solution by iteration (with $T(1) = 1$):

$$T(n) = n+(n-1)+\cdots+2+T(1) = \sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2)$$

Example: subproblems of different sizes

$$T(n) = T(n/3) + T(2n/3) + n$$

- The two subproblems have **different sizes** $n/3$ and $2n/3$: the recurrence is not of the form $a\,T(n/b) + f(n)$
- Recursion tree: every complete level has total size n, hence cost n
- The shortest root-to-leaf path (always $n/3$) has length $\log_3 n$, the longest (always $2n/3$) has length $\log_{3/2} n$
- Therefore the first $\log_3 n$ levels are complete (cost $\geq n \log_3 n$), and there are at most $\log_{3/2} n + 1$ levels, each of cost at most n. Hence

$$T(n) = \Theta(n \log n)$$

Example: hypotheses on a

a **is not constant**: $T(n) = 2^n\, T(n/2) + n^n$

- The number of subproblems $a = 2^n$ **depends on** n: the Master Theorem requires a to be a constant

$a < 1$: $T(n) = 0.5\, T(n/2) + 1/n$

- Here $a = 0.5 < 1$: the theorem requires $a \geq 1$ (there must be at least one subproblem)

Example: the regularity condition fails (1/3)

$$T(n) = T(n/2) + n(2 - \cos n)$$

- $a = 1$, $b = 2$, $n^{\log_b a} = n^0 = 1$, $f(n) = n(2 - \cos n)$
- Since $1 \leq 2 - \cos n \leq 3$, we have $n \leq f(n) \leq 3n$, so

$$f(n) = \Theta(n) = \Omega\left(n^{0+\epsilon}\right) \quad \text{with } \epsilon = 1$$

- $f(n)$ is polynomially larger than $n^{\log_b a}$, and we would expect case 3

Example: the regularity condition fails (2/3)

The regularity condition fails. Take an even $n = 2m$. Then

$$\frac{a\, f(n/2)}{f(n)} = \frac{m\,(2 - \cos m)}{2m\,(2 - \cos 2m)} = \frac{2 - \cos m}{2\,(2 - \cos 2m)}$$

- If $\cos m \leq -0.9$, then $2 - \cos m \geq 2.9$ and $\cos 2m = 2\cos^2 m - 1 \geq 2 \cdot 0.81 - 1 = 0.62$, so $2(2 - \cos 2m) \leq 2.76$ and the ratio is > 1
- Since π is irrational, the values $m \bmod 2\pi$ ($m \in \mathbb{N}$) are dense in $[0, 2\pi)$, so there are **infinitely many** m with $\cos m \leq -0.9$ (e.g., $m = 3, 9, 16, 22, 28, \ldots$)
- For $m = 22$: $\cos 22 \approx -0.99996$, $\cos 44 \approx 0.99984$, and the ratio is ≈ 1.4997

Example: the regularity condition fails (3/3)

- Hence there is no constant $c < 1$ such that $af(n/2) \leq cf(n)$ for all large n: **case 3 cannot be applied**
- Nevertheless the recurrence can be solved directly (with $n/2$ meaning $\lfloor n/2 \rfloor$): unfolding it,

$$n \leq f(n) \leq T(n) \leq \sum_{j \geq 0} 3 \frac{n}{2^j} + T(1) = 6n + T(1),$$

so

$$T(n) = \Theta(n)$$

Summary

The theorem cannot be applied when:

- $f(n)$ falls in a gap: $2T(n/2) + n/\log_2 n = \Theta(n \log\log n)$
- the recurrence is not in standard form: $T(n-1) + n = \Theta(n^2)$
- the subproblems have different sizes: $T(n/3) + T(2n/3) + n = \Theta(n \log n)$
- a is not a constant, or $a < 1$
- the regularity condition fails: $T(n/2) + n(2 - \cos n) = \Theta(n)$

Other techniques:

- recursion tree, iteration, substitution method

Proof of the Master Theorem

Introduction: Proof of the Master Theorem

- In this section we prove the theorem; the proof explains **why** the three cases hold

- We follow the classical proof, which has two parts:
  1. we first assume that n is an **exact power of** b, i.e., $n = b^L$ for some $L \in \mathbb{N}$; then all the subproblem sizes $n/b, n/b^2, \ldots$ are exact powers of b and no floor or ceiling is needed (three lemmas)
  2. we then extend the result to every natural number n, when n/b is $\lceil n/b \rceil$ or $\lfloor n/b \rfloor$

Setting (1/2)

- Throughout the proof we set

  $p = \log_b a$, so that $b^p = a$ and $p \geq 0$ (since $a \geq 1$)

- For n exact power of b we consider the recurrence

$$T(n) = \begin{cases} d & \text{if } n = 1, \\ a\,T(n/b) + f(n) & \text{if } n = b^i,\ i \geq 1, \end{cases} \tag{1}$$

  where $d > 0$ is a constant (this is the meaning of $T(1) = \Theta(1)$)

Setting (2/2)

- We assume $f(m) > 0$ for every $m \geq 1$
  - this is always true when f counts steps of an algorithm
  - in general, since f is asymptotically positive, only finitely many values are affected and only the constants of the proof change
- The hypotheses of the three cases hold "for all sufficiently large n", i.e., for $n \geq n_0$
- The following observation allows us to use them for **every** exact power of b at least b

Observation 1: from large powers to all powers

Let $f(m) > 0$ and $h(m) > 0$ for every exact power $m \geq b$ of b. If there are constants $c > 0$ and n_0 such that $f(m) \leq c\, h(m)$ (resp. $f(m) \geq c\, h(m)$) for all exact powers $m \geq n_0$, then there is a constant $c' > 0$ such that $f(m) \leq c'\, h(m)$ (resp. $f(m) \geq c'\, h(m)$) for ***all*** *exact powers $m \geq b$ of b.*

Proof

There are only finitely many exact powers of b in the interval $[b, n_0)$. Let

$$c' = \max\left\{c, \max_{b \le m < n_0} \frac{f(m)}{h(m)}\right\}$$

$$\left(\text{resp. } c' = \min\left\{c, \min_{b \le m < n_0} \frac{f(m)}{h(m)}\right\} > 0\right),$$

where m ranges over exact powers of b. Then $f(m) \le c'h(m)$ (resp. $\ge$):

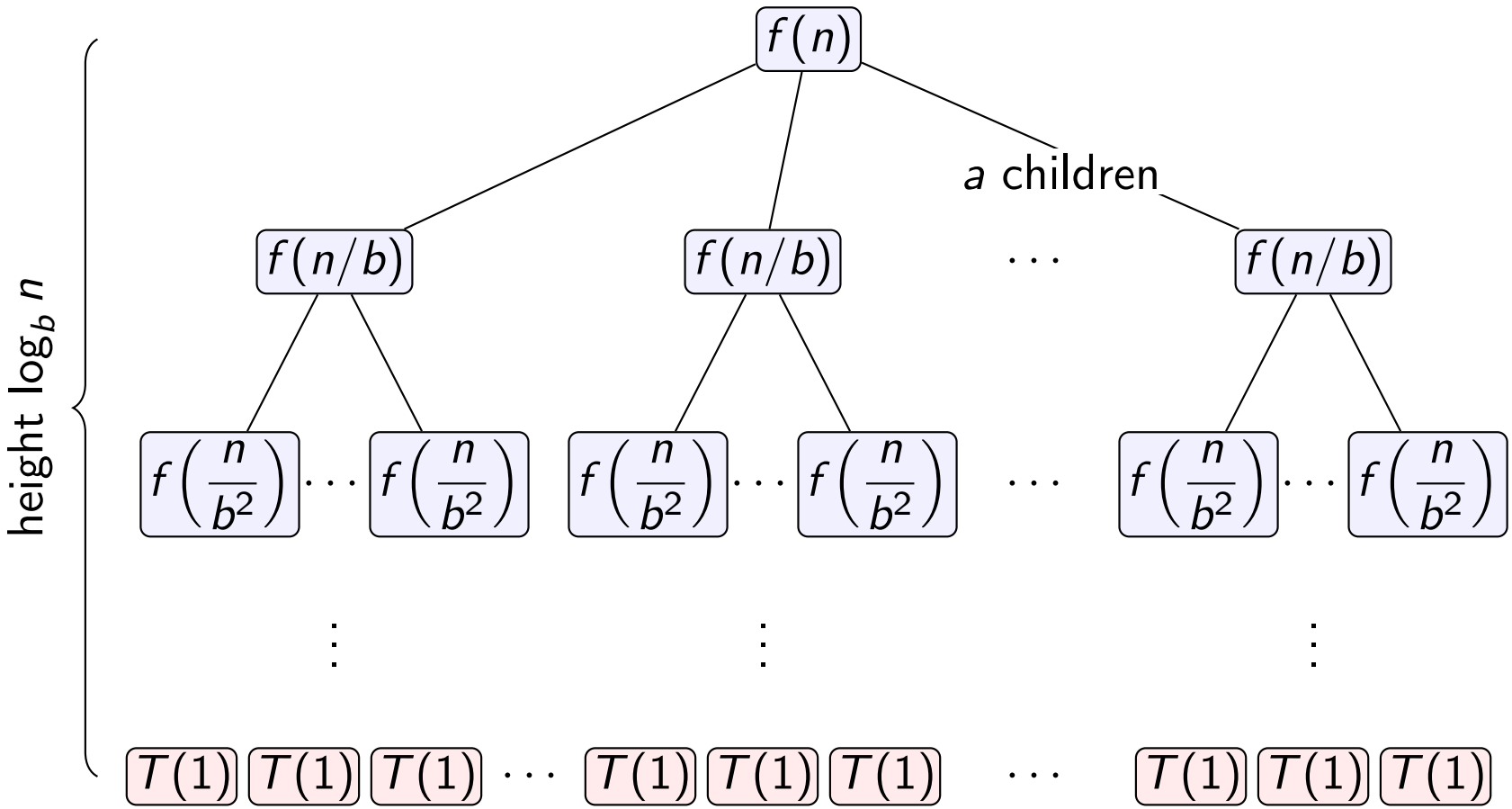
- for $m < n_0$, by the choice of c'
- for $m \ge n_0$, since $c \le c'$ (resp. $c \ge c'$)

□

- We will apply it with $h(m) = m^{p-\epsilon}$ and $h(m) = m^p \log_b^k m$, which are positive for $m \ge b$ (note $\log_b m \ge 1$ for $m \ge b$)

We now draw the recursion tree of the recurrence and compute its total cost.

The recursion tree (1/2)



The recursion tree (2/2)

Recursion tree of $T(n) = aT(n/b) + f(n)$ for n exact power of b: each internal node has a children; level j has a^j nodes, each a subproblem of size n/b^j with cost $f(n/b^j)$; the leaves are at depth $\log_b n$.

depth	**number of nodes**	**cost of the level**
0	1	$f(n)$
1	a	$af(n/b)$
2	a^2	$a^2 f(n/b^2)$
$\vdots$	$\vdots$	$\vdots$
$\log_b n$	$a^{\log_b n}$	$\Theta\left(n^{\log_b a}\right)$

total: $\Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$

Reading the tree

- The root (level 0) is a problem of size n; it costs $f(n)$ and generates a subproblems of size n/b
- By induction, level j contains a^j nodes, each a subproblem of size n/b^j and cost $f(n/b^j)$: the total cost of level j is

$$a^j f(n/b^j)$$

- The subproblem size reaches 1 when $n/b^j = 1$, i.e., at depth $j = \log_b n$: the leaves are the $a^{\log_b n}$ nodes of level $\log_b n$, each with cost $T(1) = d$

Lemma 1: Cost of the recursion tree

Let $a \geq 1$ and $b > 1$ be constants and let $f(n) > 0$ for $n \geq 1$. For every n exact power of b, the solution of recurrence (1) is

$$T(n) = d\, n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

$$= \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right).$$

Proof (1/3)

Let $n = b^L$. We first prove by induction on $i \in \{0, 1, \ldots, L\}$ that

$$T(n) = a^i \, T\left(\frac{n}{b^i}\right) + \sum_{j=0}^{i-1} a^j f\left(\frac{n}{b^j}\right). \tag{2}$$

- For $i = 0$ the sum is empty and (2) reads $T(n) = T(n)$
- Assume (2) holds for some $i < L$. Since $n/b^i = b^{L-i}$ with $L - i \geq 1$, the recurrence gives

$$T(n/b^i) = a \, T(n/b^{i+1}) + f(n/b^i)$$

Proof (2/3)

Substituting in (2):

$$T(n) = a^i \left(a\, T\left(\frac{n}{b^{i+1}}\right) + f\left(\frac{n}{b^i}\right) \right) + \sum_{j=0}^{i-1} a^j f\left(\frac{n}{b^j}\right)$$

$$= a^{i+1}\, T\left(\frac{n}{b^{i+1}}\right) + \sum_{j=0}^{i} a^j f\left(\frac{n}{b^j}\right),$$

which is (2) for $i+1$.

For $i = L = \log_b n$ we have $T(n/b^L) = T(1) = d$, hence

$$T(n) = d\, a^{\log_b n} + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Proof (3/3)

It remains to show that $a^{\log_b n} = n^{\log_b a}$. Writing $a = b^{\log_b a}$ and $n = b^{\log_b n}$,

$$\begin{aligned} a^{\log_b n} &= \left(b^{\log_b a}\right)^{\log_b n} \\ &= b^{\log_b a \cdot \log_b n} \\ &= \left(b^{\log_b n}\right)^{\log_b a} \\ &= n^{\log_b a} \end{aligned}$$

Finally, $d > 0$ is a constant, so $d\, n^{\log_b a} = \Theta(n^{\log_b a})$. □

Leaves and internal nodes

- The first term is the cost of the **leaves**: there are exactly $n^{\log_b a}$ leaves
- The sum is the cost of the **internal nodes** (levels $0, \ldots, \log_b n - 1$)

Let us check the lemma on a small instance.

Example: the lemma for $T(n) = 2T(n/2) + n$ (1/2)

We take $T(1) = 1$ and $n = 8$. Here $a = 2$, $b = 2$, $f(n) = n$, $d = 1$, $\log_2 8 = 3$.

level j	nodes a^j	size n/b^j	cost per node	cost of the level
0	1	8	$f(8) = 8$	8
1	2	4	$f(4) = 4$	8
2	4	2	$f(2) = 2$	8
3 (leaves)	$8 = 8^{\log_2 2}$	1	$T(1) = 1$	8
			total	32

Example: the lemma for $T(n) = 2T(n/2) + n$ (2/2)

- The lemma gives

$$T(8) = 1 \cdot 8^{\log_2 2} + (8 + 8 + 8) = 8 + 24 = 32$$

- Directly from the recurrence:

$$\begin{aligned} T(2) &= 2 \cdot 1 + 2 = 4, \\ T(4) &= 2 \cdot 4 + 4 = 12, \\ T(8) &= 2 \cdot 12 + 8 = 32. \quad \checkmark \end{aligned}$$

Bounding the sum

- By the lemma on the cost of the recursion tree, to solve the recurrence we need the order of growth of

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

- A **key identity**, used many times: since $b^p = a$,

$$a^j \left(\frac{n}{b^j}\right)^p = \frac{a^j}{(b^p)^j} n^p = \frac{a^j}{a^j} n^p = n^p \qquad \text{for every } j \qquad (3)$$

- In the sum, $j \leq \log_b n - 1$, so the sizes $m = n/b^j$ are exact powers of b with $m \geq b$: by the observation «from large powers to all powers», the bounds on f of cases 1 and 2 can be used for all of them

Lemma 2: Bounds on the sum

Let $a \geq 1$ and $b > 1$ be constants, $p = \log_b a$, let $f(m) > 0$ for $m \geq 1$, and let $g(n)$ be defined, for n exact power of b, as above. Then:

1. *if $f(n) = O(n^{p-\epsilon})$ for some constant $\epsilon > 0$, then $g(n) = O(n^p)$;*
2. *if $f(n) = \Theta(n^p \log^k n)$ for some constant $k \geq 0$, then $g(n) = \Theta(n^p \log^{k+1} n)$;*
3. *if $f(n) = \Omega(n^{p+\epsilon})$ for some constant $\epsilon > 0$, and $a\, f(n/b) \leq c\, f(n)$ for some constant $c < 1$ and all exact powers $n \geq n_0$, then $g(n) = \Theta(f(n))$.*

Proof of case 1 (1/2)

By the observation «from large powers to all powers» there is a constant $c > 0$ such that $f(m) \leq c\, m^{p-\epsilon}$ for every exact power $m \geq b$. Hence, using (3),

$$g(n) \leq c \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{p-\epsilon}$$

$$= c \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{p} \left(\frac{b^j}{n}\right)^{\epsilon}$$

$$= c\, n^{p-\epsilon} \sum_{j=0}^{\log_b n - 1} \left(b^{\epsilon}\right)^j$$

Proof of case 1 (2/2)

This is a finite geometric series with ratio $b^\epsilon > 1$ (since $b > 1$ and $\epsilon > 0$):

$$\sum_{j=0}^{\log_b n - 1} \left(b^\epsilon\right)^j = \frac{\left(b^\epsilon\right)^{\log_b n} - 1}{b^\epsilon - 1} = \frac{\left(b^{\log_b n}\right)^\epsilon - 1}{b^\epsilon - 1} = \frac{n^\epsilon - 1}{b^\epsilon - 1}$$

Therefore

$$0 < g(n) \leq c\, n^{p-\epsilon}\, \frac{n^\epsilon - 1}{b^\epsilon - 1} \leq \frac{c}{b^\epsilon - 1}\, n^p,$$

and since $\frac{c}{b^\epsilon - 1}$ is a positive constant, $g(n) = O(n^p)$. □

Example: case 1 of the lemma

- Recurrence $T(n) = 16\,T(n/4) + n$, with $n = 64$. Here $p = \log_4 16 = 2$, $f(m) = m = m^{2-1}$, so $c = 1$, $\epsilon = 1$
- The internal levels cost

$$g(64) = 1 \cdot 64 + 16 \cdot 16 + 256 \cdot 4 = 64 + 256 + 1024 = 1344$$

- The costs grow by the factor $b^\epsilon = 4$ from one level to the next (they are dominated by the deepest levels)
- The formula of the proof gives exactly $n^{p-\epsilon}\frac{n^\epsilon - 1}{b^\epsilon - 1} = 64 \cdot \frac{63}{3} = 1344$, and the bound $\frac{c}{b^\epsilon - 1} n^p = \frac{4096}{3} \approx 1365.3$ holds
- Adding the leaves, $T(64) = 4096 + 1344 = 5440$ with $T(1) = 1$

Proof of case 2 (1/3)

Since $\log^k n$ and $\log_b^k n$ differ by a constant factor, the hypothesis is $f(n) = \Theta(n^p \log_b^k n)$. By the observation «from large powers to all powers» there are constants $c_1, c_2 > 0$ such that

$$c_1 \, m^p \log_b^k m \leq f(m) \leq c_2 \, m^p \log_b^k m$$

for every exact power $m \geq b$. Let $L = \log_b n$. For the term j of the sum, $m = n/b^j$ and $\log_b m = L - j$; by (3),

$$a^j \left(\frac{n}{b^j}\right)^p \log_b^k \frac{n}{b^j} = n^p \, (L - j)^k$$

Proof of case 2 (2/3)

Hence, setting $i = L - j$ (as j goes from 0 to $L-1$, i goes from L down to 1),

$$c_1\, n^p\, S_k(L) \;\le\; g(n) \;\le\; c_2\, n^p\, S_k(L), \qquad \text{where} \quad S_k(L) = \sum_{i=1}^{L} i^k$$

We now prove that $\left(\frac{L}{2}\right)^{k+1} \le S_k(L) \le L^{k+1}$, i.e., $S_k(L) = \Theta(L^{k+1})$:

- *upper bound*: each of the L terms satisfies $i^k \le L^k$ (since $k \ge 0$), so $S_k(L) \le L \cdot L^k = L^{k+1}$
- *lower bound*: keep only the terms with $i \ge \lceil L/2 \rceil$ (all terms are nonnegative). There are $L - \lceil L/2 \rceil + 1 \ge L/2$ of them and each is at least $(L/2)^k$. Hence $S_k(L) \ge \frac{L}{2}\left(\frac{L}{2}\right)^k = \left(\frac{L}{2}\right)^{k+1}$

Proof of case 2 (3/3)

Therefore

$$\frac{c_1}{2^{k+1}}\, n^p \log_b^{k+1} n \;\leq\; g(n) \;\leq\; c_2\, n^p \log_b^{k+1} n,$$

that is,

$$g(n) = \Theta(n^p \log_b^{k+1} n) = \Theta(n^p \log^{k+1} n)$$

□

Example: case 2 of the lemma, $n = 16$

Recurrences $2T(n/2) + n$ and $2T(n/2) + n \log_2 n$. Here $p = \log_2 2 = 1$ and $L = \log_2 16 = 4$.

- $f(n) = n$ $(k = 0)$: every level costs $2^j \cdot \frac{16}{2^j} = 16 = n^p$, so

  $$g(16) = 4 \cdot 16 = 64 = n\,S_0(4), \qquad \text{with } S_0(4) = 4 = L$$

- $f(n) = n \log_2 n$ $(k = 1)$: level j costs $n^p(L - j) = 16\,(4 - j)$:

  $$g(16) = 64 + 48 + 32 + 16 = 160 = 16\,(4 + 3 + 2 + 1) = n\,S_1(4)$$

  The bounds of the proof give
  $\left(\frac{L}{2}\right)^2 = 4 \le S_1(4) = 10 \le L^2 = 16$, hence $64 \le g(16) \le 256$

Proof of case 3 (1/5)

Lower bound. All terms of $g(n)$ are positive and the term $j = 0$ is $f(n)$: hence $g(n) \geq f(n)$.

Upper bound. Let $L = \log_b n$. We first prove, by induction on j, that

$$a^j f\left(\frac{n}{b^j}\right) \leq c^j f(n) \qquad \text{for every } j \geq 0 \text{ such that } \frac{n}{b^{j-1}} \geq n_0 \quad (4)$$

- For $j = 0$ it is an equality
- Let $j \geq 1$ with $n/b^{j-1} \geq n_0$. Then also $n/b^{j-2} \geq n_0$, so the induction hypothesis holds for $j - 1$

Proof of case 3 (2/5)

Applying the regularity condition to the exact power $n/b^{j-1} \geq n_0$,

$$\begin{aligned} a^j f\left(\frac{n}{b^j}\right) &= a^{j-1}\left[a\, f\left(\frac{n/b^{j-1}}{b}\right)\right] \\ &\leq a^{j-1}\, c\, f\left(\frac{n}{b^{j-1}}\right) \\ &\leq c \cdot c^{j-1} f(n) = c^j f(n) \end{aligned}$$

We split $g(n)$ into the terms that satisfy $n/b^{j-1} \geq n_0$ (*top* levels) and the others (*bottom* levels).

Proof of case 3 (3/5)

Top levels: by (4) and the infinite geometric series with ratio $0 < c < 1$,

$$\sum_{\text{top}} a^j f\left(\frac{n}{b^j}\right) \leq f(n) \sum_{j=0}^{\infty} c^j = \frac{f(n)}{1-c}$$

Proof of case 3 (4/5)

- **Bottom levels**: if $n/b^{j-1} < n_0$, then $m = n/b^j < n_0$ is an exact power of b with $m \geq b$, so $f(m) \leq M$, where M is the maximum of f over the finitely many exact powers of b in $[b, n_0)$
  - moreover $a^j \leq a^L = n^p$ (lemma on the cost of the recursion tree)
  - the number of bottom levels is at most $h = \lceil \log_b n_0 \rceil$ (they correspond to $m = b, b^2, \ldots$ with $m < n_0$)

  Hence

$$\sum_{\text{bottom}} a^j f\left(\frac{n}{b^j}\right) \leq h\, M\, n^p$$

Proof of case 3 (5/5)

Finally, $f(n) = \Omega(n^{p+\epsilon})$ implies $n^p = O(f(n))$ (indeed $n^p \leq n^{p+\epsilon}$). Therefore

$$g(n) \leq \frac{f(n)}{1-c} + hMn^p = O(f(n)),$$

and together with the lower bound

$$g(n) = \Theta(f(n))$$

□

Example: case 3 of the lemma

- Recurrence $T(n) = 8T(n/3) + n^2$, with $n = 27$. Here $p = \log_3 8 \approx 1.893$, $c = \frac{8}{9}$ and the regularity condition holds for every n (so $n_0 = 1$, and there are no bottom levels)

- Level j costs $8^j \left(\frac{27}{3^j}\right)^2 = 729 \left(\frac{8}{9}\right)^j$:

$$g(27) = 729 + 648 + 576 = 1953,$$

$$f(27) = 729 \leq g(27) \leq \frac{f(27)}{1 - 8/9} = 6561$$

- The level costs decrease geometrically (they are dominated by the root)

- With $T(1) = 1$: $T(27) = 8^3 + 1953 = 512 + 1953 = 2465$

With the two lemmas we can now prove the theorem when n is an exact power of b.

Lemma 3: Master Theorem for exact powers

The three cases of the Master Theorem hold for recurrence (1), i.e., when n ranges over the exact powers of b.

Proof

By the lemma on the cost of the recursion tree, $T(n) = \Theta(n^p) + g(n)$, with $g(n) > 0$. We use the lemma on the bounds on the sum.

- Case 1: $g(n) = O(n^p)$, hence $\Theta(n^p) \leq T(n) \leq \Theta(n^p) + O(n^p)$, i.e., $T(n) = \Theta(n^p) = \Theta(n^{\log_b a})$
- Case 2: $g(n) = \Theta(n^p \log^{k+1} n)$. Since $n^p = O(n^p \log^{k+1} n)$, the leaves are negligible: $T(n) = \Theta(n^p \log^{k+1} n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- Case 3: $g(n) = \Theta(f(n))$. Since $f(n) = \Omega(n^{p+\epsilon})$, we have $n^p = O(f(n))$, so again the leaves are negligible: $T(n) = \Theta(f(n))$



It remains to remove the assumption that n is an exact power of b:
floors and ceilings.

Floors and ceilings (1/3)

- We now consider every natural number n. We treat in detail the recurrence with ceilings,

$$\begin{aligned} T(n) &= a\,T\left(\lceil n/b \rceil\right) + f(n) && \text{for } n \geq n', \\ d_1 \leq T(n) &\leq d_2 && \text{for } 1 \leq n < n', \end{aligned} \tag{5}$$

  where n' and $0 < d_1 \leq d_2$ are constants, and $f(n) > 0$ for every $n \geq 1$

- In case 3 the regularity condition reads $a\,f(\lceil n/b \rceil) \leq c\,f(n)$ for all $n \geq n_0$

Floors and ceilings (2/3)

- **Enlarging the threshold.** The values $T(m)$ for $1 \leq m < N$ are finitely many positive numbers for every constant $N \geq n'$. Hence, replacing d_1 by their minimum and d_2 by their maximum, we may assume that the recurrence is used only for $n \geq N$, where we choose

$$N \geq \max \left\{ n', \ n_0, \ \frac{2b-1}{b-1} \right\}$$

  Then the hypotheses on f (and the regularity condition) hold for every argument $\geq N$

Floors and ceilings (3/3)

- **Sizes of the subproblems.** At depth j of the recursion tree all subproblems have the same size n_j, where

$$n_0^* = n, \qquad n_j = \lceil n_{j-1}/b \rceil \quad (j \geq 1)$$

(we write n_0^* for the size at depth 0 to avoid confusion with the threshold n_0)

Lemma 4: Sizes with ceilings

For every $j \geq 0$:

$$\frac{n}{b^j} \leq n_j < \frac{n}{b^j} + \frac{b}{b-1}.$$

Proof

By induction on j. For $j = 0$, $n_0^* = n$. For $j \geq 1$, using $x \leq \lceil x \rceil < x + 1$ and the induction hypothesis,

$$n_j \geq \frac{n_{j-1}}{b} \geq \frac{n}{b^j},$$

$$\begin{aligned} n_j &< \frac{n_{j-1}}{b} + 1 \\ &< \frac{1}{b}\left(\frac{n}{b^{j-1}} + \frac{b}{b-1}\right) + 1 \\ &= \frac{n}{b^j} + \frac{1}{b-1} + 1 = \frac{n}{b^j} + \frac{b}{b-1} \end{aligned}$$

□

Lemma 5: Master Theorem with ceilings

The three cases of the Master Theorem hold for recurrence (5), for every natural number n.

- The proof has four steps: the depth of the tree, the unfolding, the bounds on the sum of the internal levels, the conclusion

Proof (1/7)

Let $n \geq N$ and $L = \log_b n$ (a real number, not necessarily an integer).

Step 1: depth of the tree. Let D be the smallest j such that $n_j < N$ (the depth at which the recursion stops).

- For $j \geq L$, the lemma on the sizes with ceilings gives $n_j < 1 + \frac{b}{b-1} = \frac{2b-1}{b-1} \leq N$; hence $D \leq \lceil L \rceil < L + 1$
- For $j \leq L - \log_b N$, $n_j \geq n/b^j \geq N$; hence $D > L - \log_b N$

Thus

$$L - \log_b N < D < L + 1$$

Proof (2/7)

Step 2: unfolding. Exactly as in the proof of the lemma on the cost of the recursion tree (the recurrence holds at every size $n_j \geq N$, i.e., for $j < D$),

$$T(n) = a^D \, T(n_D) + \sum_{j=0}^{D-1} a^j f(n_j)$$

Since $d_1 \leq T(n_D) \leq d_2$ and, by Step 1,

$$\frac{n^p}{N^p} = a^{L - \log_b N} \leq a^D \leq a^{L+1} = a \, n^p,$$

the first term is $\Theta(n^p)$.

Proof (3/7)

Step 3: bounding $g(n) = \sum_{j=0}^{D-1} a^j f(n_j)$. Let $m_j = n/b^j$. For $0 \le j \le D-1$ we have $j < L$, so $m_j > 1$, and by the lemma on the sizes with ceilings

$$m_j \le n_j < m_j + \frac{b}{b-1} < C\, m_j, \qquad C = \frac{2b-1}{b-1}, \tag{6}$$

while $n_j \ge N$, so the bounds on f hold at n_j. Recall $a^j m_j^p = n^p$ by (3).

Proof (4/7)

- Case 1: $f(n_j) \leq c\, n_j^{p-\epsilon} \leq c\, C'\, m_j^{p-\epsilon}$ with $C' = \max\{1, C^{p-\epsilon}\}$ (if $p - \epsilon \geq 0$ use $n_j < Cm_j$, otherwise $n_j \geq m_j$). As in the lemma on the bounds on the sum,

$$
\begin{aligned}
g(n) &\leq cC' n^{p-\epsilon} \sum_{j=0}^{D-1} (b^\epsilon)^j \\
&\leq cC' n^{p-\epsilon} \frac{b^{\epsilon D}}{b^\epsilon - 1} \\
&\leq cC' n^{p-\epsilon} \frac{b^\epsilon n^\epsilon}{b^\epsilon - 1} = O(n^p)
\end{aligned}
$$

Proof (5/7)

- Case 2: **Upper bound**:
  $\log_b n_j < \log_b C + \log_b m_j \leq \log_b C + L$, so

  $$a^j f(n_j) \leq c_2 a^j C^p m_j^p (L + \log_b C)^k = c_2 C^p n^p (L + \log_b C)^k,$$

  and summing the $D < L + 1$ terms,

  $$g(n) \leq c_2 C^p n^p (L + 1)(L + \log_b C)^k = O(n^p \log^{k+1} n)$$

Proof (6/7)

- Case 2: **Lower bound**: $n_j \geq m_j > 1$ gives

$$a^j f(n_j) \geq c_1 a^j m_j^p \log_b^k m_j = c_1 n^p (L-j)^k$$

For n large enough $D - 1 \geq L/2$ (because $D > L - \log_b N$), so keeping only the terms $0 \leq j \leq L/2$ (at least $L/2$ terms, each $\geq (L/2)^k$),

$$g(n) \geq c_1 n^p \left(\frac{L}{2}\right)^{k+1} = \Omega(n^p \log^{k+1} n)$$

Proof (7/7)

- Case 3: $g(n) \geq f(n)$ (term $j = 0$). For $1 \leq j \leq D-1$ we have $n_{j-1} \geq N \geq n_0$, so the regularity condition gives

$$a\, f(n_j) = a\, f(\lceil n_{j-1}/b \rceil) \leq c\, f(n_{j-1});$$

by induction (as in (4)) $a^j f(n_j) \leq c^j f(n)$, hence $g(n) \leq \frac{f(n)}{1-c}$ and $g(n) = \Theta(f(n))$

Step 4: conclusion. $T(n) = \Theta(n^p) + g(n)$ with the same bounds on $g(n)$ as in the lemma on the bounds on the sum; the three cases follow exactly as in the proof of the Master Theorem for exact powers. □

Floors (1/2)

- For $T(n) = a\,T(\lfloor n/b \rfloor) + f(n)$ the proof is the same, with $n_j = \lfloor n_{j-1}/b \rfloor$ and the bounds

$$\frac{n}{b^j} - \frac{b}{b-1} < n_j \leq \frac{n}{b^j}$$

(proved as the lemma on the sizes with ceilings, using $x - 1 < \lfloor x \rfloor \leq x$)

Floors (2/2)

- Choosing also $N \geq \max\{b, \frac{2b}{b-1}\}$, for $j < D$ we get $m_j \geq n_j \geq N$, hence $\frac{b}{b-1} \leq \frac{m_j}{2}$ and $\frac{m_j}{2} < n_j \leq m_j$, which replaces (6)
- Moreover $L - \log_b\left(N + \frac{b}{b-1}\right) \leq D \leq L - \log_b N + 1$. Only the constants change
- This completes the proof of the Master Theorem. ■

Example: merge sort for every n (1/3)

$$T(n) = T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor) + n, \qquad T(1) = 1$$

- This is the exact recurrence of merge sort on an array of any length n
- It is not of the form $aT(\lceil n/b \rceil) + f(n)$, but it is **sandwiched** between two recurrences of that form:

$$U(n) = 2\,U(\lceil n/2 \rceil) + n, \qquad W(n) = 2\,W(\lfloor n/2 \rfloor) + n,$$

  with $U(1) = W(1) = 1$
- U and W are nondecreasing (by strong induction: if $n < n''$ then $\lceil n/2 \rceil \leq \lceil n''/2 \rceil$ and the added term n is smaller)

Example: merge sort for every n (2/3)

- By strong induction, $W(n) \leq T(n) \leq U(n)$: for $n \geq 2$, using that U is nondecreasing and $\lfloor n/2 \rfloor \leq \lceil n/2 \rceil$,

  $$T(n) \leq U(\lceil n/2 \rceil) + U(\lfloor n/2 \rfloor) + n \leq 2\, U(\lceil n/2 \rceil) + n = U(n),$$

  and symmetrically $T(n) \geq 2\, W(\lfloor n/2 \rfloor) + n = W(n)$

Example: merge sort for every n (3/3)

- By the Master Theorem with ceilings (case 2, $k = 0$) and its floor version, $U(n) = \Theta(n \log n)$ and $W(n) = \Theta(n \log n)$. Hence $T(n) = \Theta(n \log n)$ for **every** n, not only for powers of 2
- Numerically: $T(1000) = 10976$ and $T(10^6) = 20951424$, with $\frac{T(n)}{n \log_2 n} \approx 1.101$ and ≈ 1.051, respectively

The regularity condition

- In case 3 the hypothesis $f(n) = \Omega(n^{p+\epsilon})$ is in fact implied by the regularity condition (for positive f)
- This explains why, in practice, one only has to check the regularity condition

Observation 2: regularity implies polynomial growth

Let $a \geq 1$, $b > 1$, $p = \log_b a$, *and let* $f(m) > 0$ *for every exact power* m *of* b. *If* $a\, f(n/b) \leq c\, f(n)$ *for some constant* $0 < c < 1$ *and for all exact powers* $n \geq n_0$ *of* b, *then*

$$f(n) = \Omega\left(n^{p+\epsilon}\right)$$

with

$$\epsilon = \log_b \frac{a}{c} - \log_b a = -\log_b c > 0.$$

Proof (1/3)

Since $0 < c < 1$, $\epsilon = -\log_b c > 0$ and $\frac{a}{c} > a \geq 1$. The regularity condition can be rewritten as

$$f(n) \geq \frac{a}{c} f\left(\frac{n}{b}\right) \qquad \text{for every exact power } n \geq n_0$$

Proof (2/3)

- Let $n \geq n_0$ be an exact power of b, and let $j \geq 0$ be the number of times we can apply this inequality, i.e., the smallest j such that $m = n/b^j$ satisfies $m < b\,n_0$
- So $n_0 \leq m < b\,n_0$ when $n \geq n_0$; every intermediate size n/b^i, $i < j$, is $\geq bn_0 \geq n_0$

Applying the inequality j times,

$$f(n) \geq \left(\frac{a}{c}\right)^j f\left(\frac{n}{b^j}\right) \geq \left(\frac{a}{c}\right)^j \mu,$$

where $\mu > 0$ is the minimum of f over the finitely many exact powers of b in $[n_0, bn_0)$.

Proof (3/3)

Since $b^j = n/m > \frac{n}{bn_0}$ and $\frac{a}{c} = b^{\log_b(a/c)}$ with $\log_b(a/c) > 0$,

$$\left(\frac{a}{c}\right)^j = \left(b^j\right)^{\log_b(a/c)} > \left(\frac{n}{bn_0}\right)^{\log_b(a/c)}$$

Finally $\log_b(a/c) = \log_b a - \log_b c = p + \epsilon$, hence

$$f(n) \geq \frac{\mu}{(bn_0)^{p+\epsilon}}\, n^{p+\epsilon} \qquad \text{for every exact power } n \geq n_0,$$

that is, $f(n) = \Omega(n^{p+\epsilon})$. □

Two remarks on the observation

- For example, in the example $8\,T(n/3) + n^2$ ($c = 8/9$) the observation gives

$$\epsilon = -\log_3 \frac{8}{9} = 2 - \log_3 8 \approx 0.107,$$

  exactly the largest ϵ found there

- The converse is false: in the example $T(n/2) + n(2 - \cos n)$, $f(n) = \Omega(n^{p+1})$ but the regularity condition does not hold

Summary (1/2)

Exact powers of b:

- recursion tree: $T(n) = d\, n^{\log_b a} + g(n)$, with $g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$
- key identity: $a^j (n/b^j)^p = n^p$, with $p = \log_b a$

Bounds on the sum:

- case 1: geometric series with ratio $b^\epsilon > 1$, $g(n) = O(n^p)$
- case 2: $g(n) = \Theta(n^p S_k(L))$ with $S_k(L) = \Theta(L^{k+1})$
- case 3: $f(n) \leq g(n) \leq f(n)/(1-c) + hMn^p$, $g(n) = \Theta(f(n))$

Summary (2/2)

Every natural number n:

- with ceilings, $n/b^j \leq n_j < n/b^j + b/(b-1)$; the depth D differs from $\log_b n$ by a constant
- the same bounds hold, only the constants change; floors are analogous
- recurrences sandwiched between two standard ones (merge sort with any n)

Regularity condition:

- it implies $f(n) = \Omega(n^{p+\epsilon})$ with $\epsilon = -\log_b c$
- the converse is false

Summary: how to solve a recurrence $T(n)$ (1/2)

1. Is it in standard form $T(n) = a\,T(n/b) + f(n)$ with constants $a \geq 1$, $b > 1$ and $f(n)$ asymptotically positive?
   - **No** $\longrightarrow$ use another method: iteration, recursion tree, substitution
   - **Yes** $\longrightarrow$ compute $n^{\log_b a}$ and compare it with $f(n)$

2. $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$ (polynomially smaller)?
   $\longrightarrow$ case 1: $T(n) = \Theta(n^{\log_b a})$

3. $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$?
   $\longrightarrow$ case 2: $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$

Summary: how to solve a recurrence $T(n)$ (2/2)

4. $f(n) = \Omega(n^{\log_b a+\epsilon})$ for some $\epsilon > 0$ (polynomially larger)? Check the regularity condition $a\,f(n/b) \leq c\,f(n)$ with $c < 1$:
   - holds $\longrightarrow$ case 3: $T(n) = \Theta(f(n))$
   - fails $\longrightarrow$ the theorem does not apply

5. None of the above (a **gap**, e.g., $f(n) = n^{\log_b a}/\log n$) $\longrightarrow$ the theorem does not apply: use the recursion tree

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n-1} a^j f(n/b^j)$$

Base of the logarithms: irrelevant in $\log^k n$, essential in $n^{\log_b a}$.